

AI大模型开发39CNN二

0.课程内容

0.1 晨考

0.2 课程回顾

Pytorch

实现全连接神经网络

务必掌握使用流程和概念

1.数据

2.数据处理

3.数据集 (Dataset DataLoader)

4.定义模型

全连接 激活函数

前向传播

5.训练

train()

循环多次

循环遍历 DataLoader

前向传播

计算损失

反向传播

优化器参数更新

.....

6.评测/评估

eval

不需要梯度计算

前向传播

计算损失

准确率

7.持久化

save

CNN 卷积神经网络，视觉领域，处理图片

核心：

1.卷积层

2.激活函数

3.池化层

4.全连接

基于手写数字实现CNN的全过程

1.CNN

1.1 CNN

卷积神经网络 (Convolutional Neural Network, CNN) 是一种专门处理具有网格拓扑结构数据（如图像）的深度学习模型。它的核心思想源于人类视觉皮层的工作机制——局部感受野和权值共享。

处理图像相关，全连接和CNN的差异：

问题	全连接网络	CNN
参数量	224×224×3 图像 → 隐藏层 1000 神经元 = 1.5 亿参数	同等感受野仅需 几百个参数
空间信息	展平后丢失空间结构	保留空间局部关系
平移不变性	无	通过卷积和池化获得

1.2 核心

1.卷积层

卷积层：Convolution Layer是 CNN 的核心，通过**卷积核 (filter/kernel)** 在输入上滑动，提取局部特征。

卷积操作是卷积层的核心，它通过一个称为**卷积核 (或滤波器)** 的矩阵与输入数据（通常是图像）进行局部加权求和。

具体步骤如下：

- **输入数据**：通常是图像，可以表示为一个三维数组（例如，高度 × 宽度 × 颜色通道），对于灰度图像则为二维数组（高度 × 宽度）。
- **卷积核**：一个小的矩阵，用于在输入数据上**滑动并提取特征**。卷积核的大小（如2x2、3x3、5x5等）和数量（即滤波器的数量）是可配置的。

- **步长 (Stride)**：卷积核在输入数据上移动的步长。步长为1时，卷积核每次移动一个单位；步长为2时，每次移动两个单位。
- **填充 (Padding)**：在输入数据的（一圈4个边）边缘添加的零值，用于控制输出数据的尺寸。例如，如果希望输出数据的尺寸与输入相同，可以添加适当的零填充。

- padding=1 加一圈0
- padding=0 不加
- padding=same 强制保证输出的矩阵和输入矩阵大小一样，自动补0
- 为什么填充：CNN扫描，如果不填充，有时会出现边缘数据丢失
- 输入：3*3

1 2 3

1 2 3

1 2 3

0 0 0 0 0

0 1 2 3 0

0 1 2 3 0

0 1 2 3 0

0 0 0 0 0

步长=2

卷积核=2

没有填充

输出=1/2+1

向上取整或舍弃

第一次：

1 2

1 2

第二次：

3 0

3 0

第三次：

1 2

0 0

输出=1

输入数据：3*3

1 2 3

4 5 6

7 8 9

卷积核: 2*2

1 0

0 1

步长=1

填充没有

经过卷积层计算处理, 新矩阵的大小=(3-2)/1 +1=2 ==>2*2

输出尺寸计算公式

$$O = \lfloor \frac{I-K+2P}{S} \rfloor + 1$$

其中:

- O = 输出尺寸
- I = 输入尺寸
- K = 卷积核大小
- P = padding 大小
- S = stride 步长

新矩阵O(2*2)

计算

左上角: $o[0][0]=6$

从输入层起点开始 ($[0][0]$) 圈定一个和卷积核大小相同的

计算的输入矩阵:

1 2

4 5

卷积核:

1 0

0 1

2个矩阵内积就是结果= $1*1+2*0+4*0+5*1=6$

右上角: $o[0][1]=8$

移动的步长为1, 往右移动一次, 重新圈定新矩阵

计算的输入矩阵:

2 3

5 6

在和卷积核进行内积= $2*1+3*0+5*0+6*1=8$

左下角: $o[1][0]=12$
下一行从起点开始 $[1][0]$
计算输入矩阵:
4 5
7 8
在和卷积核进行内积 $=4*1+5*0+7*0+8*1=12$
右下角: $o[1][1]=$
下一行从起点开始 $[1][1]$
计算输入矩阵:
5 6
8 9
在和卷积核进行内积 $=5*1+6*0+8*0+9*1=14$
经过卷积层输出的新矩阵 $o=$
6 8
12 14

2.池化层

池化层用于**降低特征图空间维度**，减少参数量和计算量，同时增强平移不变性。

池化层有2种形式：

1.Max Pool 取最大值

2.Avg Pool 取均值

输入矩阵: $4*4$

```
1 2 3 4
5 6 7 8
1 2 3 4
5 6 7 8
```

池化要求: 窗口 $2*2$ 步长=2 采用Max Pool

新矩阵: $(4-2)/2+1=2 \rightarrow 2*2$

```
o[0][0]=6
1 2
5 6
max=6
o[0][1]=8
3 4
7 8
max=8
.....
6 8
6 8

16*16---> 2*2
16-8 步长2
8-4 步长2
```

3.激活函数-ReLU

4.全连接

5.学习率

6.批处理数量

7.正则化-Dropout

2.OpenCV

2.1 OpenCV

OpenCV开源的图像识别处理的库，支持Python等语言

识别加载图像并对图像做处理

在计算机眼里，一张图片是什么？

就是一个由数字组成的多维数组（NumPy 数组）。

一张彩色图片通常被表示为一个三维数组：（高度，宽度，通道数）。比如（1080，1920，3），分别代表这张图有 1080 个像素高、1920 个像素宽，并且每个像素点由红(R)、绿(G)、蓝(B)三个颜色通道的值组成。

2.2 初体验

1.下载

pip install opencv-python

2.使用

```
# 导入 OpenCV
import cv2
# 插班版本号
print("版本号: ",cv2.__version__)
```

3.加载图片

```
# 导入 OpenCV
import cv2
# 插班版本号
print("版本号: ",cv2.__version__)

# 加载图片 把图片转换为 数组
data1=cv2.imread("grill.png")
print(data1)
```

2.3 核心操作

图片读取

图片变为灰图

图片裁剪

图片缩放

图片显示

图片保存

示例代码:

```
# 导入 openCV
import cv2
# 插班版本号
print("版本号: ",cv2.__version__)

# 加载图片 把图片转换为 数组
data1=cv2.imread("gril1.png")
print(data1)

# 灰图
data2=cv2.imread("gril1.png",cv2.IMREAD_GRAYSCALE)
print(data2)
# 显示图片
cv2.imshow("gril1",data2)
# 加个等待 窗口持续显示
cv2.waitKey(0)
# 图片信息
print(data2.shape)
print(data2.dtype)

# 保存图片
cv2.imwrite("cvgril1.png",data2)
```

```
import cv2

# 图片操作
# 参数说明: IMREAD_COLOR 带颜色 IMREAD_GRAYSCALE 灰图
data=cv2.imread("gril1.png")
print(data)
# 裁剪 获取部分数据
data1=data[100:400,100:400,:]
# 查看效果
cv2.imshow("gril1裁剪",data1)
# 不按0就一直显示
cv2.waitKey(0)

# 缩放
```

```
data2=cv2.resize(data,(100,100))
cv2.imshow("resize",data2)
cv2.waitKey(0)
```

```
# 边缘检测 Canny
data3=cv2.Canny(data,150,200)
print(data3)
cv2.imshow("canny",data3)
cv2.waitKey(0)
cv2.imwrite("canny.jpg",data3)
cv2.destroyAllWindows()
```

3.YOLO

3.1 YOLO

YOLO:You Only Look Once

机器视觉领域最主流的框架

3.2 初体验

下载库

pip install ultralytics

导入

```
from ultralytics import YOLO

print("版本号: ",ultralytics.__version__)
```

识别图片内容

```
import ultralytics
from ultralytics import YOLO

print("版本号: ",ultralytics.__version__)

# 创建模型对象
model=YOLO("yolov8n.pt")
# print(model)
# 加载图片 做目标检测
r1=model("grill.png")
# print("识别结果: ",r1)
r1[0].save(filename='yologrill.png')

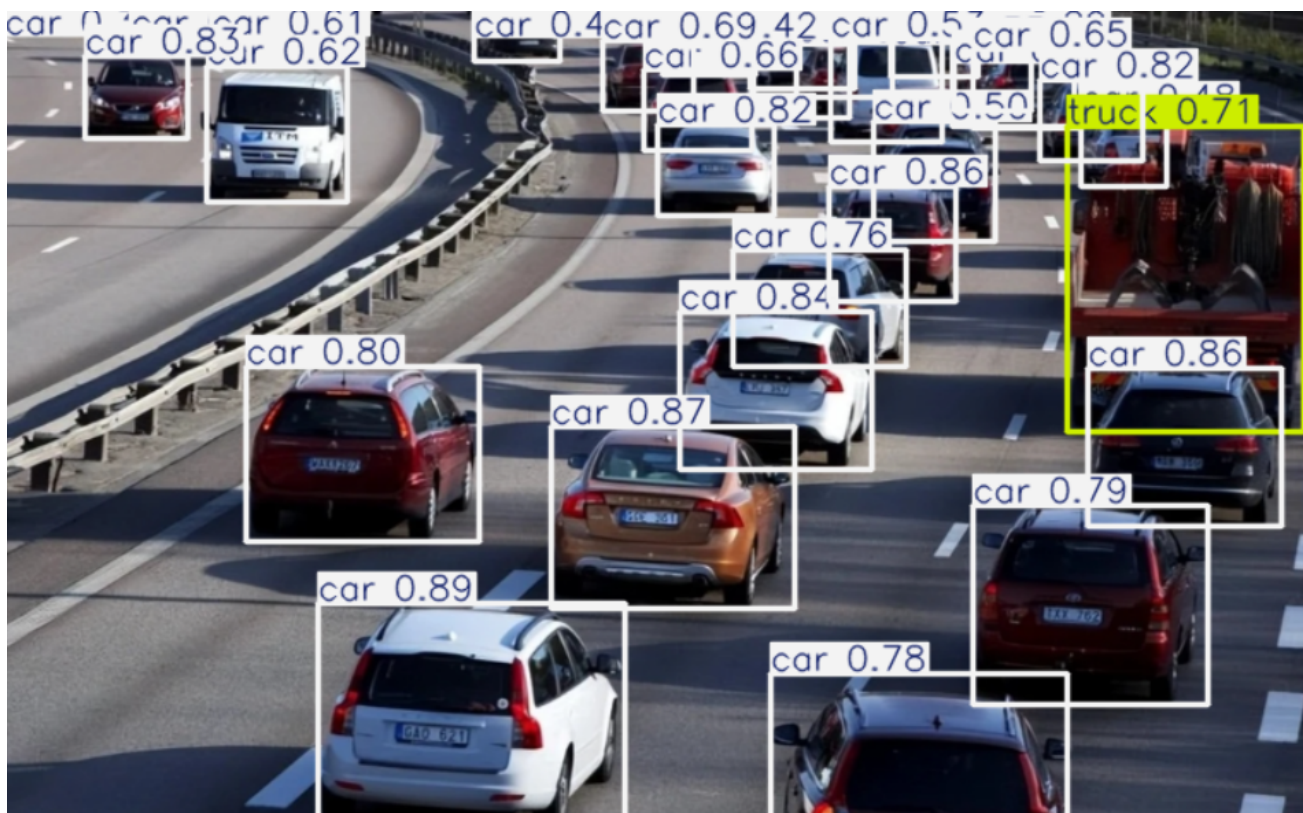
r2=model("pic1.png")
r2[0].save(filename='y1pic1.png')
```



```

r3=model("pic.png")
r3[0].save(filename='y1pic.png')
r4=model("pic2.png")
r4[0].save(filename='y1pic2.png')
r5=model("pic4.png")
r5[0].save(filename='y1pic4.png')

```



3.3 核心

可以结合视频、摄像头进行实时检测

结合OpenCV实现自动检测

```

import cv2
import torch
from ultralytics import YOLO
import datetime

# =====
# 1. 初始化配置
# =====
# 检查是否有可用的 GPU (CUDA), 没有则用 CPU
device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
print(f"使用设备: {device}")

# 加载 YOLOv8 模型 (首次运行会自动下载 yolov8n.pt)
# 如果你想检测特定类别, 可以去官网查 COCO 数据集的类别索引
# 0: person, 2: car, 16: dog 等
model = YOLO('yolov8n.pt').to(device)

```

```

# =====
# 2. 开启摄像头
# =====
cap = cv2.VideoCapture(0) # 0 代表默认摄像头

if not cap.isOpened():
    print("错误: 无法打开摄像头! ")
    exit()

print("摄像头已开启, 按 'q' 退出, 按 's' 保存当前帧截图...")

while True:
    # 读取一帧画面
    ret, frame = cap.read()
    if not ret:
        print("无法读取画面, 退出...")
        break

    # =====
    # 3. 使用 YOLO 进行推理
    # =====
    # verbose=False 关闭命令行打印的检测日志, 让界面更干净
    results = model(frame, stream=True, verbose=False)

    # 初始化当前帧的人数
    person_count = 0

    # =====
    # 4. 解析结果并用 OpenCV 绘制
    # =====
    for result in results:
        boxes = result.boxes # 获取检测框信息

        for box in boxes:
            # 获取坐标 (xyxy格式: 左上角x, 左上角y, 右下角x, 右下角y)
            x1, y1, x2, y2 = map(int, box.xyxy[0])

            # 获取置信度
            conf = float(box.conf[0])

            # 获取类别 ID 和名称
            cls_id = int(box.cls[0])
            cls_name = model.names[cls_id]

            # 我们只关心“人” (person), COCO数据集中 person 的 id 是 0
            if cls_id == 0 and conf > 0.5: # 置信度大于 50% 才算有效
                person_count += 1

            # 用 OpenCV 画矩形框 (BGR颜色: 绿色)
            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

            # 构造标签文本

```

```

label = f'{cls_name} {conf:.2f}'

# 用 OpenCV 写文字
cv2.putText(frame, label, (x1, y1 - 10),
             cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

# =====
# 5. 显示统计信息与画面
# =====
# 在画面左上角显示人数
info_text = f'Persons: {person_count}'
cv2.putText(frame, info_text, (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

# 显示处理后的画面
cv2.imshow('YOLOv8 Real-time Detection', frame)

# =====
# 6. 键盘交互
# =====
key = cv2.waitKey(1) & 0xFF

# 按 'q' 退出
if key == ord('q'):
    break
# 按 's' 保存当前截图
elif key == ord('s'):
    fn=datetime.datetime.now().strftime('%Y%m%d%H%M%S')+'_screenshot.jpg'
    cv2.imwrite("ims/"+fn, frame)
    print(f"截图已保存为{fn}")

# 释放资源
cap.release()
cv2.destroyAllWindows()

```

4. 迁移学习

迁移学习 (Transfer Learning) 才是工业界的主流——把一个在百万张图片上训练好的“超级大脑”，通过微调，变成你自己的专用工具。

迁移学习的本质是基于一个现成模型，只需要更换模型一些信息：

1. 分类

2. 全连接层

重新训练，得出新模型，这个新模型在这个新数据领域上的表现相对较好

比如基于ResNet模型实现猫狗分类的单独训练

5.总结

1.CNN

卷积神经网络 图片 视觉

手算：卷积层

手算：池化层

基于CNN+PyTorch实现一个图片分类的

2.OpenCV

图像处理的开源库

转换图像、裁剪、缩放、保存.....

3.YOLO

工业级主流的CNN的模型

图片、视频、摄像头

基于YOLO实现新数据的训练

4.迁移学习

基于现成的模型进行二次开发

1.冻结（不用进行梯度计算）原有的特征参数

2.数据处理

3.模型的微改 全连接层

4.训练和评估

5.新模型

6.作业

1.计算

输入

1 2 2

2 3 3

3 4 4

卷积核：

1 0

0 1

步长1

2. OpenCV 图像预处理脚本**

请编写一个 Python 脚本，完成以下任务：

1. 使用 `cv2.imread` 读取一张你喜欢的图片。
2. 将图片转换为**灰度图**。
3. 将图片**缩放到** 300x300 像素。
4. 使用 `cv2.Canny` 对缩放后的灰度图进行边缘检测。
5. 将最终结果保存为 `result.png` 并显示出来。

YOLO 初体验与识别

1. 找一张包含“人”和“汽车”的街景图片，使用 YOLO 进行识别。
2. 将识别后的结果（带框的图片）保存到本地，并观察控制台输出的 `boxes` 数据，指出如何获取“人”的置信度（Confidence）。